

The R1 Recipe in Code

Module 3 — Five stages, three notebooks

One toy task — **1-digit addition** — runs through every stage.

We watch **one number** climb the whole way: the task **pass-rate**.

The map: 5 stages → 3 notebooks

Notebook	Stages it covers	The point
01 · Foundations	base model + why a scratchpad helps	intermediate tokens <i>measurably</i> help
02 · The RL core	cold-start SFT → GRPO	teach the format, then reward correctness
03 · Amplify & compress	rejection-SFT → distillation	self-improve, then shrink

Each notebook runs top-to-bottom on a **laptop CPU** in minutes.

Notebook 01 — Foundations

Every reasoning model starts as a plain **next-token predictor**.

Then the key experiment: train two tiny models on addition — one answers **directly**, one gets a **<think> scratchpad**.

```
notebooks/01_foundations_reasoning_and_cot.ipynb
```

The insight that justifies the whole course

The scratchpad model wins.

Intermediate tokens aren't decoration — they're **computation**.

That single result is *why* reasoning models exist.

Notebook 02 — The RL core (1/2): Cold-start SFT

Teach the **format** first: `<think>...</think><answer>...</answer>` .

A handful of examples locks in the shape — answers still mostly wrong.

Pass-rate: low single digits. That's expected.

Notebook 02 — The RL core (2/2): GRPO

Sample the same prompt **G times**, score each by rule, push toward the group's best.

No critic. No value network. No human labels — **the group is the baseline**.

Watch the pass-rate climb.

```
notebooks/02_rl_core_coldstart_sft_and_grpo.ipynb
```

Notebook 03 — Amplify (1/2): Rejection-sampling SFT

Use the RL'd model as a **data factory**: generate → keep only correct → fine-tune.

The pass-rate jumps again with **no new training signal** — just better data.

Notebook 03 — Compress (2/2): Distillation

Pour the skill into a **smaller student** with KL on the logits.

A model ~**1/7 the size** keeps most of the pass-rate.

```
notebooks/03_amplify_and_compress_rejection_sft_and_distillation.ipynb
```


The payoff — one metric, the whole recipe

Stage	What it adds	Pass-rate
Cold-start	format	—
GRPO	skill	■
Reject-SFT	stability	■
Distill	efficiency	■ (smaller)

(Exact numbers print live in the notebook — that's the demo.)

This is still how it's done

The same recipe — **GRPO + RL from verifiable rewards** — is how 2026 frontier reasoning models (GPT-5.5, Claude Opus 4.8, Gemini 3.x) are trained.

| Scaled up massively. **Not replaced.**

Bonus → picking & proving in practice

- `notebooks/07_picking_a_reasoning_model_for_an_application.ipynb`
a multi-provider bake-off with an LLM judge
- `notebooks/08_reproducibility_cheap_vs_flagship.ipynb`
fix a **seed**, match a flagship with a **cheaper** model, count the cost

The training story, then the **engineering** story.